

Let $a \geq 1$, $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence.

$T(n) = aT(n/b) + f(n)$. Then

- 1) If $f(n) = O(n^{\log_b(a) - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b(a)})$
- 2) If $f(n) = \Theta(n^{\log_b(a)})$, then $T(n) = \Theta(n^{\log_b(a)} \lg n)$
- 3) If $f(n) = \Omega(n^{\log_b(a) + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$

Use the following to find complexity class for

1) $T(n) = 4T(n/2) + n$ $a=4, b=2, f(n)=n$
 $n = O(n^{\log_2 4 - \epsilon})$

$n = O(n^{2 - \epsilon}), \epsilon > 0$

case 1

$T(n) = \Theta(n^2)$

2) $T(n) = 4T(n/2) + n^2$ $a=4, b=2, f(n)=n^2$
 $n^2 = O(n^{\log_2 4 - \epsilon}), \epsilon = 0$

case 2

$T(n) = \Theta(n^2 \lg n)$

3) $T(n) = 4T(n/2) + n^3$

$n^3 = O(n^{\log_2 4 - \epsilon}), \epsilon < 0$

case 3

$af(n/b) \leq cf(n)$ for some $c < 1$

$f(n) = n^3$

$4\left(\frac{n}{2}\right)^3 \leq cn^3$
 $\frac{1}{2}n^3 \leq cn^3$
 $c = \frac{1}{2}$

$T(n) = \Theta(n^3)$

$D = \text{Merge}(B, C, \text{size_B}, \text{size_C})$

Merge takes two sorted lists in nondecreasing order B, C, and creates one sorted list D

Write pseudocode to solve this problem.

What is the running time of Merge in terms of size_B and size_C?

size B + size C

Use Merge to create Merge-Sort using recursion. Merge-Sort takes as input A, first, last and splits the list in two equal halves, sorts each half, and then merges the two halves back together.

Write Pseudocode for merge-sort?

Write a recurrence relation for the running time for Merge-Sort?

Solve the recurrence relation with Masters Method.

$$2T(n/2) + n, \quad a=2, b=2, f(n)=n$$

$$n = O(n^{1-\epsilon}), \quad \epsilon > 0$$

case 2